

General Project Report — Building AI Agent

Introduction

Building AI Agent is a progressive project designed to build an AI agent from scratch.

The project does not start with an LLM or with external frameworks.

Instead, it starts from the basic logic of an agent and evolves step by step toward a more professional architecture.

The goal is not only to create working code.

The goal is to understand how an agent is designed, how responsibilities are separated, and how a simple notebook can evolve into a real project structure.

Project Goal

The goal of the project is to build an agent capable of:

- receiving a user request;
- understanding the user's intent;
- selecting the correct tool;
- extracting the required arguments;
- validating the data;
- executing the selected tool;
- storing the interaction state;
- evolving toward a modular and professional architecture.

The project is also designed to teach the core concepts behind modern AI agents.

These concepts include:

- routing;
 - parsing;
 - execution;
 - memory;
 - planning;
 - tool registry;
 - LLM-based routing;
 - LLM-based parsing;
 - multi-step reasoning;
 - professional project structure.
-

Development Philosophy

The project follows one simple rule:

one version, one concept

Each version introduces only one important idea.

This keeps the learning process clear and controlled.

Instead of building a complex agent immediately, the project grows naturally:

```
simple agent
↓
separated components
↓
generic components
↓
planning
↓
managed memory
↓
tool registry
↓
LLM integration
↓
multi-step behavior
↓
professional project structure
```

This approach helps build strong foundations before introducing advanced features.

General Architecture

The agent is designed as a pipeline of components.

A simplified flow is:

```
User
↓
Router
↓
Parser
↓
Executor
```

↓
Memory

Each component has a clear responsibility.

The Router chooses the tool.

The Parser extracts arguments.

The Executor validates and runs the selected tool.

The Agent coordinates the flow.

The Memory stores the interaction state.

This separation is the foundation of a clean and professional architecture.

Router

The Router decides which tool should be used.

It analyzes the user request and identifies the most suitable action.

Example:

What is 2 + 3?

should be routed to:

"addition"

At the beginning, the Router can be rule-based and use keywords or synonyms.

Later, it can evolve into an LLM-based Router.

The Router answers the question:

Which tool should be used?

Parser

The Parser converts the user request into structured arguments.

Example:

```
What is 2 + 3?
```

becomes:

```
{"a": 2, "b": 3}
```

Another example:

```
Hello, my name is luca.
```

becomes:

```
{"name": "luca"}
```

The Parser is the bridge between natural language and Python functions.

Its natural evolution is to move from rigid rules to a Generic Parser and later to an LLM Parser.

Executor

The Executor is responsible for running the selected tool.

It receives:

```
tool_name  
arguments
```

Then it validates the arguments and executes the correct function.

The Executor exists to keep execution logic separate from the Agent.

The rule is:

```
Agent = coordinates  
Executor = validates and executes
```

This makes the system easier to maintain and extend.

Agent

The Agent is the main coordinator of the system.

Its role is not to do everything directly.

Instead, it orchestrates the other components:

```
receive request
↓
select tool
↓
parse arguments
↓
execute tool
↓
save state
```

A well-designed Agent should remain clean.

It should coordinate specialized components instead of containing all the logic itself.

Memory

Memory stores the state of each interaction.

A typical state looks like this:

```
{
  "request": "What is 2 + 3?",
  "action": "addition",
  "arguments": {"a": 2, "b": 3},
  "result": 5
}
```

At first, memory can be a simple list.

Later, it can evolve into a Memory Manager with functions such as:

```
save_to_memory(state)
get_last_interaction()
clear_memory()
```

This makes memory easier to manage and reuse.

Planner

The Planner introduces a more advanced behavior.

Its job is to create a small plan before execution.

Example:

```
{
  "tool": "addition",
  "reason": "Add two numbers.",
  "needs_arguments": ["a", "b"]
}
```

The Planner helps the agent reason before acting.

This is an important step toward multi-step agents.

The Planner answers the question:

```
What should the agent do before executing?
```

Tool Registry

The Tool Registry manages tools in a cleaner and more scalable way.

Instead of manually writing all tools inside a dictionary, the project can evolve toward a registration system:

```
register_tool(...)
```

This makes it easier to:

- add new tools;
- manage metadata;
- keep descriptions organized;
- avoid large manual dictionaries;
- scale the system.

The Tool Registry becomes important when the number of tools grows.

LLM Router

A later version can introduce an LLM-based Router.

The LLM can help choose the correct tool from more natural user requests.

Example:

Can you calculate the total of five and seven?

could be routed to:

"addition"

even if the request does not contain a simple `+` symbol.

The LLM should not be introduced too early.

It should improve an already clear architecture, not hide a weak one.

LLM Parser

An LLM Parser can extract arguments from more natural language.

Example:

Can you add five and seven?

can become:

```
{  
  "a": 5,  
  "b": 7  
}
```

This allows the agent to handle more flexible and realistic user requests.

The rule is:

```
use rules first to understand the system  
use LLMs later to handle natural language complexity
```

Multi-step Agent

A more advanced agent should be able to perform multiple steps.

Example:

```
Add 2 and 3, then multiply the result by 4
```

can become:

```
1. addition(2, 3)
2. multiplication(result, 4)
```

This introduces multi-step reasoning and tool orchestration.

The agent no longer performs only one action.

It creates and executes a sequence of actions.

Roadmap

The complete roadmap is:

```
V1  → Agent base
V2  → Executor
V3  → Generic Parser
V4  → Planner
V5  → Memory Manager
V6  → Tool Registry
V7  → LLM Router
V8  → LLM Parser
V9  → Multi-step Agent
V10 → Professional Project Structure
```

Each version adds one specific architectural concept.

This keeps the project easy to understand and easy to grow.

Final Professional Structure

The project starts in a notebook because notebooks are useful for learning, testing, and explaining.

Later, the project can evolve into a professional file structure:

```
tools.py
router.py
parser.py
executor.py
memory.py
planner.py
agent.py
```

Each file contains one responsibility.

This makes the project:

- easier to read;
- easier to test;
- easier to maintain;
- easier to extend;
- closer to a real professional project.

Educational Value

The project teaches more than coding.

It teaches how to think about agent architecture.

The main concepts are:

- separation of responsibilities;
- modular design;
- incremental development;
- test-driven improvements;
- controlled use of LLMs;
- tool-based execution;
- memory management;
- planning;
- transition from notebook to real project.

Each component is introduced to solve a specific design problem.

The Executor exists because the Agent should not execute tools directly.

The Generic Parser exists because rigid parsing logic does not scale.

The Planner exists because an agent should reason before acting.

The Memory Manager exists because memory should be managed, not only stored.

The Tool Registry exists because tools should be organized and described clearly.

Conclusion

Building AI Agent is a structured learning project for building an AI agent from scratch.

Its strength is controlled evolution.

Each version adds one new concept and improves the architecture without creating confusion.

The final result is not only an agent that works, but a system designed with clarity, modularity, and professional structure.

This is the transition from a simple script to a real agent project.